

Des LLM aux agents locaux

Formation Robotech — du fonctionnement interne au modèle local, puis à son harnais

Ahmed

<https://github.com/Robotech-Lillois>

30 septembre 2026



©️ Ce support est mis à disposition sous licence **CC BY-NC-SA 4.0** — figures : 3Blue1Brown

Sommaire

- 1 Introduction
- 2 Comprendre un LLM
- 3 Faire tourner un modèle
- 4 Harnais d'agents
- 5 Exercice
- 6 Conclusion

Ce que vous saurez faire dans 2 heures

- 1 **Expliquer** ce qui se passe dans un LLM quand il « parle »
(tokens → plongements → attention → mot suivant)
- 2 **Faire tourner** un modèle localement avec llama.cpp
(choisir taille, quantification et contexte selon votre machine)
- 3 **Comprendre un agent** : la différence entre un chat et une boucle d'agent, et brancher un modèle local sur un harnais (DeepSeek Harness)

Prérequis : ligne de commande (la formation Git suffit). Aucune maths au-delà du lycée.

Pourquoi faire tourner des modèles en local?

Local

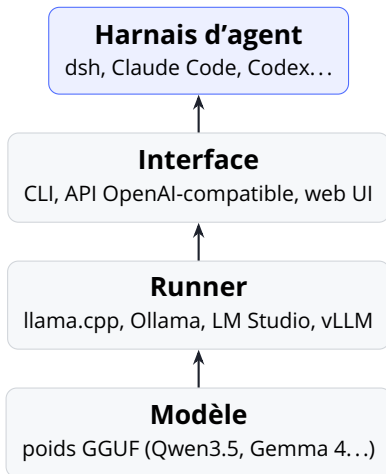
- Confidentialité : les données ne sortent pas
- Coût marginal nul, pas de limite de débit
- Fonctionne hors ligne (demos, terrain)
- Contrôle et compréhension : excellent pour apprendre

Cloud / API

- Capacité brute (les très gros modèles)
- Aucune machine à dimensionner
- Raisonnement avancé, multimodal large

Ce n'est pas une religion, c'est un **compromis** — et aujourd'hui le local est réellement utile.

La pile logicielle en une image



- Ollama et LM Studio sont llama.cpp sous le capot
- Même modèle, même quantification, même GPU $\Rightarrow$ la vitesse ne change pas, seuls les wrappers diffèrent
- Le dénominateur commun : **l'API compatible OpenAI**

Le fil rouge de la session

agent = modèle + harnais

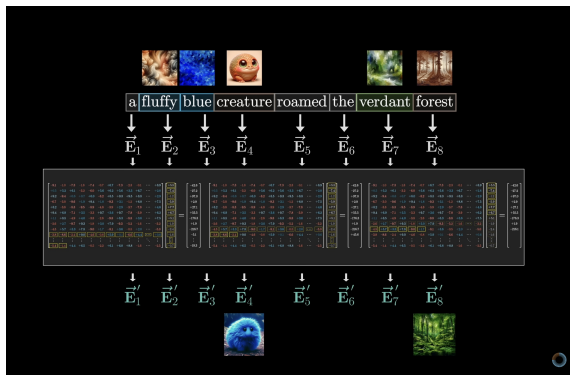
- Un LLM est un **prédicteur du token suivant** : sans mémoire, sans outils, il ne produit que du texte
- Tout le reste (chat, outils, actions sur plusieurs étapes) est du **logiciel autour** : c'est le harnais
- Programme : **1.** ce qui se passe dans le modèle — **2.** le faire tourner chez soi — **3.** l'agent autour

La prédiction du token suivant

- Objectif du modèle : étant donné un texte, **prédire ce qui vient ensuite**
- Le texte est d'abord découpé en **tokens** — souvent des morceaux de mots, pas des mots entiers
- Le modèle établit un **classement de tous les mots possibles** : ceux qui collent au contexte ont plus de chances de sortir
- On tire dans ce classement (plus ou moins au hasard), puis on recommence ⇒ une « conversation »

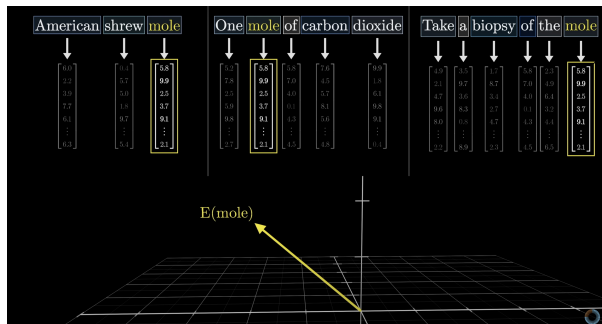


Les plongements (embeddings)



- Chaque token devient une **longue liste de nombres** — un vecteur — : c'est son **plongement** (embedding)
- Certains aspects du sens correspondent à des **directions** dans cet espace (ex. king et queen partagent une direction « genre »)
- À l'entrée, cette liste ne connaît que le token — pas encore son contexte

Un mot, plusieurs sens

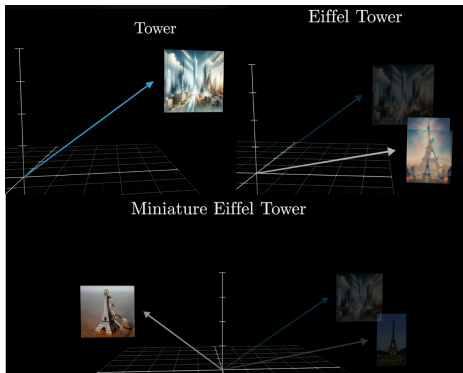


mole : American shrew mole / one mole of CO₂ / a biopsy of the mole

Avant l'attention, le vecteur de « mole » est identique dans les trois phrases.

Il faut un mécanisme pour que les voisins passent de l'information au vecteur.

L'information voyage loin



tower + Eiffel → Paris, fer ... puis +
miniature → plus du tout « grand et
haut ».

Et pour finir un roman policier par
« therefore the murderer was ... » :
le dernier vecteur doit avoir
absorbé **tout le contexte
pertinent**.

L'attention : requêtes, clés, valeurs

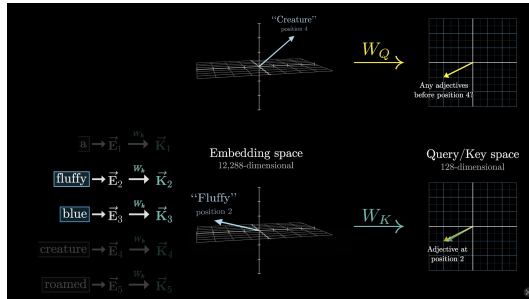
	$\vec{E}_1$	$\vec{E}_2$	$\vec{E}_3$	$\vec{E}_4$	$\vec{E}_5$	$\vec{E}_6$	$\vec{E}_7$	$\vec{E}_8$
$\vec{E}_1 \xrightarrow{W_1} \vec{V}_1$	1.00 $\vec{V}_1$	0.00 $\vec{V}_1$	0.00 $\vec{V}_1$	0.00 $\vec{V}_1$	0.00 $\vec{V}_1$	0.00 $\vec{V}_1$	0.00 $\vec{V}_1$	0.00 $\vec{V}_1$
$\vec{E}_2 \xrightarrow{W_2} \vec{V}_2$	0.00 $\vec{V}_2$	1.00 $\vec{V}_2$	0.00 $\vec{V}_2$	0.00 $\vec{V}_2$	0.00 $\vec{V}_2$	0.00 $\vec{V}_2$	0.00 $\vec{V}_2$	0.00 $\vec{V}_2$
$\vec{E}_3 \xrightarrow{W_3} \vec{V}_3$	0.00 $\vec{V}_3$	0.00 $\vec{V}_3$	1.00 $\vec{V}_3$	0.00 $\vec{V}_3$	0.00 $\vec{V}_3$	0.00 $\vec{V}_3$	0.00 $\vec{V}_3$	0.00 $\vec{V}_3$
$\vec{E}_4 \xrightarrow{W_4} \vec{V}_4$	0.00 $\vec{V}_4$	0.00 $\vec{V}_4$	0.00 $\vec{V}_4$	1.00 $\vec{V}_4$	0.00 $\vec{V}_4$	0.00 $\vec{V}_4$	0.00 $\vec{V}_4$	0.00 $\vec{V}_4$
$\vec{E}_5 \xrightarrow{W_5} \vec{V}_5$	0.00 $\vec{V}_5$	0.00 $\vec{V}_5$	0.00 $\vec{V}_5$	0.00 $\vec{V}_5$	1.00 $\vec{V}_5$	0.00 $\vec{V}_5$	0.00 $\vec{V}_5$	0.00 $\vec{V}_5$
$\vec{E}_6 \xrightarrow{W_6} \vec{V}_6$	0.00 $\vec{V}_6$	0.00 $\vec{V}_6$	0.00 $\vec{V}_6$	0.00 $\vec{V}_6$	0.00 $\vec{V}_6$	1.00 $\vec{V}_6$	0.00 $\vec{V}_6$	0.00 $\vec{V}_6$
$\vec{E}_7 \xrightarrow{W_7} \vec{V}_7$	0.00 $\vec{V}_7$	0.00 $\vec{V}_7$	0.00 $\vec{V}_7$	0.00 $\vec{V}_7$	0.00 $\vec{V}_7$	0.00 $\vec{V}_7$	1.00 $\vec{V}_7$	0.00 $\vec{V}_7$
$\vec{E}_8 \xrightarrow{W_8} \vec{V}_8$	0.00 $\vec{V}_8$	0.00 $\vec{V}_8$	0.00 $\vec{V}_8$	0.00 $\vec{V}_8$	0.00 $\vec{V}_8$	0.00 $\vec{V}_8$	0.00 $\vec{V}_8$	1.00 $\vec{V}_8$
	$\Delta \vec{E}_1$	$\Delta \vec{E}_2$	$\Delta \vec{E}_3$	$\Delta \vec{E}_4$	$\Delta \vec{E}_5$	$\Delta \vec{E}_6$	$\Delta \vec{E}_7$	$\Delta \vec{E}_8$

$\vec{E}_1$	$\vec{E}_2$	$\vec{E}_3$	$\vec{E}_4$	$\vec{E}_5$	$\vec{E}_6$	$\vec{E}_7$	$\vec{E}_8$
+	+	+	+	+	+	+	+
$\Delta \vec{E}_1$	$\Delta \vec{E}_2$	$\Delta \vec{E}_3$	$\Delta \vec{E}_4$	$\Delta \vec{E}_5$	$\Delta \vec{E}_6$	$\Delta \vec{E}_7$	$\Delta \vec{E}_8$
$\vec{E}_1'$	$\vec{E}_2'$	$\vec{E}_3'$	$\vec{E}_4'$	$\vec{E}_5'$	$\vec{E}_6'$	$\vec{E}_7'$	$\vec{E}_8'$

Une **tête d'attention** : chaque token fabrique trois petits vecteurs (des paramètres appris) :

- une **requête** : la question du type « y a-t-il un adjectif avant moi ? »
- une **clé** : ce que ce token peut offrir en réponse
- une **valeur** : l'information effectivement transmise

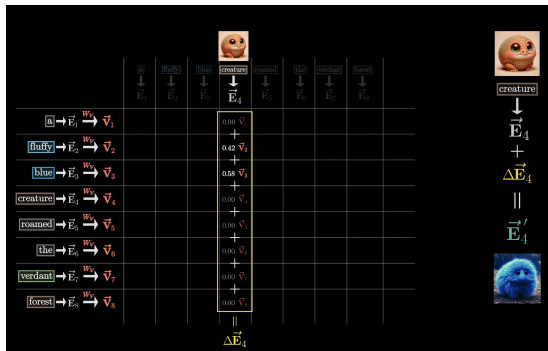
Scores de pertinence



	<code>a</code>	<code>fluffy</code>	<code>blue</code>	<code>creature</code>	<code>roamed</code>	<code>the</code>	<code>verdant</code>	<code>forest</code>	
	$\vec{E}_1$	$\vec{E}_2$	$\vec{E}_3$	$\vec{E}_4$	$\vec{E}_5$	$\vec{E}_6$	$\vec{E}_7$	$\vec{E}_8$	
	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	
	$\vec{Q}_1$	$\vec{Q}_2$	$\vec{Q}_3$	$\vec{Q}_4$	$\vec{Q}_5$	$\vec{Q}_6$	$\vec{Q}_7$	$\vec{Q}_8$	
<code>a</code> → $\vec{E}_1$ → $\vec{K}_1$	$\vec{K}_1 \cdot \vec{Q}_1$	$\vec{K}_1 \cdot \vec{Q}_2$	$\vec{K}_1 \cdot \vec{Q}_3$	$\vec{K}_1 \cdot \vec{Q}_4$	$\vec{K}_1 \cdot \vec{Q}_5$	$\vec{K}_1 \cdot \vec{Q}_6$	$\vec{K}_1 \cdot \vec{Q}_7$	$\vec{K}_1 \cdot \vec{Q}_8$	
<code>fluffy</code> → $\vec{E}_2$ → $\vec{K}_2$	$\vec{K}_2 \cdot \vec{Q}_1$	$\vec{K}_2 \cdot \vec{Q}_2$	$\vec{K}_2 \cdot \vec{Q}_3$	$\vec{K}_2 \cdot \vec{Q}_4$	$\vec{K}_2 \cdot \vec{Q}_5$	$\vec{K}_2 \cdot \vec{Q}_6$	$\vec{K}_2 \cdot \vec{Q}_7$	$\vec{K}_2 \cdot \vec{Q}_8$	
<code>blue</code> → $\vec{E}_3$ → $\vec{K}_3$	$\vec{K}_3 \cdot \vec{Q}_1$	$\vec{K}_3 \cdot \vec{Q}_2$	$\vec{K}_3 \cdot \vec{Q}_3$	$\vec{K}_3 \cdot \vec{Q}_4$	$\vec{K}_3 \cdot \vec{Q}_5$	$\vec{K}_3 \cdot \vec{Q}_6$	$\vec{K}_3 \cdot \vec{Q}_7$	$\vec{K}_3 \cdot \vec{Q}_8$	
<code>creature</code> → $\vec{E}_4$ → $\vec{K}_4$	$\vec{K}_4 \cdot \vec{Q}_1$	$\vec{K}_4 \cdot \vec{Q}_2$	$\vec{K}_4 \cdot \vec{Q}_3$	$\vec{K}_4 \cdot \vec{Q}_4$	$\vec{K}_4 \cdot \vec{Q}_5$	$\vec{K}_4 \cdot \vec{Q}_6$	$\vec{K}_4 \cdot \vec{Q}_7$	$\vec{K}_4 \cdot \vec{Q}_8$	
<code>roamed</code> → $\vec{E}_5$ → $\vec{K}_5$	$\vec{K}_5 \cdot \vec{Q}_1$	$\vec{K}_5 \cdot \vec{Q}_2$	$\vec{K}_5 \cdot \vec{Q}_3$	$\vec{K}_5 \cdot \vec{Q}_4$	$\vec{K}_5 \cdot \vec{Q}_5$	$\vec{K}_5 \cdot \vec{Q}_6$	$\vec{K}_5 \cdot \vec{Q}_7$	$\vec{K}_5 \cdot \vec{Q}_8$	
<code>the</code> → $\vec{E}_6$ → $\vec{K}_6$	$\vec{K}_6 \cdot \vec{Q}_1$	$\vec{K}_6 \cdot \vec{Q}_2$	$\vec{K}_6 \cdot \vec{Q}_3$	$\vec{K}_6 \cdot \vec{Q}_4$	$\vec{K}_6 \cdot \vec{Q}_5$	$\vec{K}_6 \cdot \vec{Q}_6$	$\vec{K}_6 \cdot \vec{Q}_7$	$\vec{K}_6 \cdot \vec{Q}_8$	
<code>verdant</code> → $\vec{E}_7$ → $\vec{K}_7$	$\vec{K}_7 \cdot \vec{Q}_1$	$\vec{K}_7 \cdot \vec{Q}_2$	$\vec{K}_7 \cdot \vec{Q}_3$	$\vec{K}_7 \cdot \vec{Q}_4$	$\vec{K}_7 \cdot \vec{Q}_5$	$\vec{K}_7 \cdot \vec{Q}_6$	$\vec{K}_7 \cdot \vec{Q}_7$	$\vec{K}_7 \cdot \vec{Q}_8$	
<code>forest</code> → $\vec{E}_8$ → $\vec{K}_8$	$\vec{K}_8 \cdot \vec{Q}_1$	$\vec{K}_8 \cdot \vec{Q}_2$	$\vec{K}_8 \cdot \vec{Q}_3$	$\vec{K}_8 \cdot \vec{Q}_4$	$\vec{K}_8 \cdot \vec{Q}_5$	$\vec{K}_8 \cdot \vec{Q}_6$	$\vec{K}_8 \cdot \vec{Q}_7$	$\vec{K}_8 \cdot \vec{Q}_8$	

Chaque requête est comparée à chaque clé $\Rightarrow$ une grille de scores : « qui est pertinent pour mettre à jour qui ? »

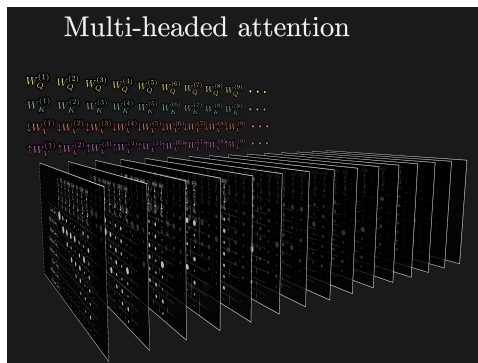
Comment un mot est mis à jour



Les valeurs des voisins, mélangées selon leur **pertinence**, sont **ajoutées au vecteur d'origine**.

creature a absorbé fluffy et blue → un sens contextualisé.

Plusieurs têtes, plusieurs blocs



- Une **attention multi-têtes** : des têtes parallèles, chacune ses propres paramètres → autant de « façons dont le contexte change le sens » (ex. grammaire vs références : Harry → sorcier ou prince?)
- Des **centaines de blocs** empilés (attention + MLP) : chaque passage affine les plongements — sentiment, ton, style...

En vrai, ça compte combien de paramètres?

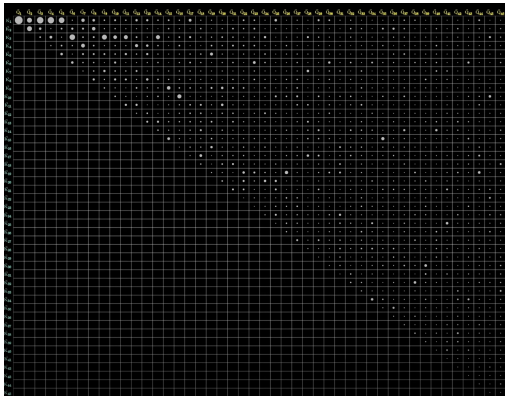
Total weights: 175,181,291,520		 GPT-3	
Organized into 27,938 matrices			
Embedding	$\overset{12,288}{d_embed} * \overset{50,257}{n_vocab} = 617,558,016$		
Key	$\overset{128}{d_query} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers} = 14,495,514,624$		
Query	$\overset{128}{d_query} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers} = 14,495,514,624$		
Value	$\overset{128}{d_value} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers} = 14,495,514,624$		
Output	$\overset{12,288}{d_embed} * \overset{128}{d_value} * \overset{96}{n_heads} * \overset{96}{n_layers} = 14,495,514,624$		
Up-projection	57,982,058,496		
Down-projection			
Unembedding	$\overset{50,257}{n_vocab} * \overset{12,288}{d_embed} = 617,558,016$		

Un ordre de grandeur pour donner envie :

- GPT-3 : 96 couches, des dizaines de têtes chacune
- L'attention représente environ **un tiers des 175 milliards de paramètres**

Tout ça, ce sont des nombres à stocker
⇒ pourquoi la quantification (partie 2) est indispensable.

Pourquoi le contexte coûte cher



- Chaque token peut interagir avec tous les autres : doubler le contexte quadruple le travail
- En génération, on garde en mémoire les clés/valeurs de chaque token : le **KV cache**
- La fenêtre de contexte est donc une vraie limite physique, pas un réglage gratuit

Prefilling et décodage

- **Prefilling** : tout le prompt est traité en parallèle (rapide, exploite le GPU) → remplit le KV cache
- **Décodage** : ensuite, un token à la fois (lent, limité par la mémoire) → c'est le tok/s affiché par llama.cpp
- **Échantillonnage** : la température est un dé plus ou moins pipé sur le tirage du mot suivant → déterminisme vs créativité

```
llm ... eval: 12.3 t/s decod | 1450 t/s prefill | ctx 8192
```

(les deux vitesses n'ont rien à voir — gardez-le en tête pour les agents, partie 3)

Ce qu'un LLM n'est pas

- **Pas une base de données** : « halluciner » = continuer plausiblement, pas rattraper une erreur de fait
- **Pas de mémoire** entre deux appels : chaque requête repart de zéro
⇒ c'est le logiciel autour qui gère l'historique (partie 3)
- **Pas de compréhension symbolique certifiable** : ce qui marche remarquablement bien, avec un modèle statistique sous le capot

Et pourtant ça marche : l'attention est **massivement parallélisable** sur GPU, et l'échelle seule a produit des sauts qualitatifs. C'est la leçon des 10 dernières années.

llama.cpp et le format GGUF

llama.cpp (projet ggml)

- Inférence LLM en C/C++ pur, sans dépendances, MIT, ~130k étoiles GitHub
- « minimal setup, state-of-the-art performance on a wide range of hardware »
- Backends : Metal (Apple), CUDA, Vulkan, HIP, CPU (AVX/NEON)...+ hybride CPU+GPU

GGUF

- Format conteneur : poids + tokenizer + gabarit de chat
- Distribué sur Hugging Face
- Une même famille existe en plusieurs **quantifications** — un fichier = un choix

La quantification

Schéma	bits/poids (env.)	Usage
Q8_0	~8,5	quasi sans perte, gros
Q4_K_M	~4,8	le défaut recommandé : l'essentiel de Q8 à mi-taille
Q3_K_M	~3,3	compromis serré, la qualité baisse vite
Q2_K_S	~2,5	dernier recours

- Le modèle stocke des nombres à 16 bits; les compresser à 4–5 bits divise la mémoire par 3–4 avec peu de perte perceptible
- Les repos « Dynamic GGUF » (Unsloth) optimisent le choix couche par couche

Dimensionner : poids + KV cache

poids ($\approx \text{\#paramètres} \times \text{bits/poids}$) + **KV cache** (contexte) $\leq$ **RAM/VRAM**

Modèle	Quant.	Poids	Machine type
0,8B	Q4_K_M	$\sim 0,5$ Go	n'importe quel laptop
4-8B	Q4_K_M	$\sim 2,5-5$ Go	laptop récent / GPU 6-8 Go
9-14B	Q4_K_M	$\sim 5-9$ Go	GPU 12 Go
35B	Q4_K_M	~ 21 Go (avec ctx 32k)	RTX 3090/4090

Le KV cache s'ajoute au-dessus, et il grandit avec le contexte. Un agent construit des prompts longs : prévoyez large (cf. exercice).

Le paysage des modèles (sept. 2026)

- **Qwen3.5** (0,8B → 35B+), **Qwen3.8-Flash-Next**, **Gemma 4** (variantes QAT), gpt-oss... — le paysage bouge vraiment vite
- Les modèles MoE (experts éparés) : efficaces mais gourmands — experts déchargés sur disque = lent
- Règle pratique : prendre la **plus grande famille que votre machine accepte en Q4**, en instruct (pas « thinking » par défaut pour les agents)
- Où chercher : Hugging Face, dépôts Unsloth; docs llama.cpp pour les archis supportées

(vérifié le 29/09/2026 — relire la liste « vérifier la veille » du plan le jour J)

Trois commandes suffisent

- 1 **Discuter dans le terminal** (télécharge depuis Hugging Face) :

```
llama cli -hf ggml-org/Qwen3.5-0.8B-GGUF
```

- 2 **Servir le modèle** (API compatible OpenAI et interface web intégrée) :

```
llama-server -hf ggml-org/Qwen3.5-0.8B-GGUF \  
--host 127.0.0.1 --port 8080 -c 8192 --jinja
```

- 3 **Comprendre la ligne d'état** : prefill $\gg$ decode en tok/s; -c = taille du contexte
 $\Rightarrow$ KV cache.

--jinja active le gabarit de chat natif du modèle — indispensable pour que les appels d'outils (partie 3) sortent au format OpenAI. Sans lui, un agent reste un chatbot.

Démo A — un modèle qui tourne chez vous (en direct)

Script

```
# 1. chat terminal
llama cli -hf ggml-org/Qwen3.5-0.8B-GGUF

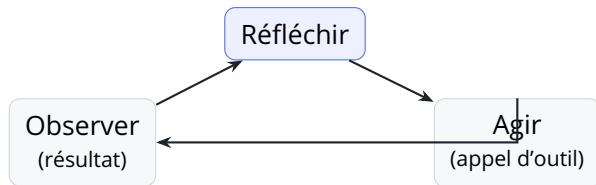
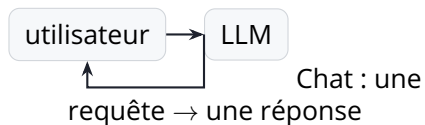
# 2. serveur + web UI
llama-server -hf ggml-org/Qwen3.5-0.8B-
GGUF \
--host 127.0.0.1 --port 8080 \
-c 8192 --jinja
# puis: http://127.0.0.1:8080
```

À regarder

- le téléchargement et la taille du fichier GGUF
- la ligne tok/s : préfilling vs décodage
- même question en Q4 puis Q8 : taille vs qualité
- l'API : GET /v1/models

Script complet : demo/demo_llama.sh (pré-télécharger les GGUF hors salle!)

Chat et agent : la boucle



Agent : boucle raisonner → agir → observer
(idée « ReAct », 2023), jusqu'à ce que la tâche soit finie

Un LLM seul ne peut que produire du texte. **Le harnais exécute** ses intentions.

L'appel d'outils (function calling)

- 1 Le serveur publie le **catalogue d'outils** (schémas JSON) : lire un fichier, chercher, exécuter...
- 2 Le modèle émet un appel structuré — ce n'est pas lui qui l'exécute :

```
tool_calls: [{"function": {"name": "read_file",  
                           "arguments": "{\\"path\\": \\"parts.csv\\"}"}}]
```

- 3 Le harnais exécute, renvoie le résultat, la boucle continue.

Côté runner, il faut que le serveur sache sérialiser ces appels : llama.cpp → --jinja.
C'est le pont entre la partie 2 et la partie 3.

Un harnais, concrètement, ça fait quoi?

agent = modèle + harnais

Le harnais gère

- la boucle et l'état (le modèle est sans mémoire)
- l'exécution des outils, le sandbox
- la gestion du contexte (fenêtre, compression)
- les garde-fous : permissions, approbations

Exemples courants

- Claude Code, Codex, Cursor (codage)
- **DeepSeek Harness (dsh)**
- harnais « interne » (livré) vs « externe » (AGENTS.md, serveurs MCP, skills — Böckeler)

Le coût du contexte

- Chaque requête repart avec : instructions système + catalogue d'outils + historique $\Rightarrow$ une tâche minuscule peut coûter **14,7K tokens d'entrée** (~25 schémas d'outils renvoyés à chaque étape — mesures 08/2026)
- En local, ce coût se paie en **temps de préfilling**, pas en euros
- Piège classique : le `contextWindow` déclaré au harnais doit correspondre au -c du serveur
 $\Rightarrow$ sinon **troncature silencieuse** du milieu du prompt (ou erreur franche selon l'archi)
- Les harnais mûrs **compriment** l'historique et mettent le contexte en cache — comme le présent support a été rédigé par une session d'agent

MCP, skills, fichiers d'instructions

- **MCP** (Model Context Protocol) : prise standardisée pour brancher des outils tiers sur n'importe quel harnais compatible
- **Skills** : dossiers `SKILL.md` découverts par le harnais — procédures réutilisables, en texte pur, versionnables avec Git
- **AGENTS.md** : le mode d'emploi du projet, lu par l'agent avant d'agir (un « guide » — à distinguer des « capteurs » : tests, linters, qui vérifient après)
- Idée commune : tout est **fichier local, lisible, versionnable** — pas de magie

DeepSeek Harness (dsh)

Ce que c'est

- harnais d'agents open source (MIT, DeepSeek AI, 2026 — « developer preview »)
- architecture « tout est un plugin » (modèles, outils, UI)
- Web UI locale : `npx @deepseek-ai/dsh web` → `127.0.0.1:3080`

Brancher un modèle local

- Settings → Models → « Add a custom model API »
- protocole `openai-completions`, base `http://127.0.0.1:8080/v1`
- clé factice acceptée mais obligatoire; id exact du modèle; `contextWindow ≤ -c`

Config fichier équivalente : `profil cordis.patch.yml` (voir `demo/dsh-provider-example.yml`)

Installer dsh sur Debian — quelques obstacles

1. Node trop vieux

- Debian installe Node trop vieux; dsh exige **Node** $\geq$ **22,19** (22.x) — Node 23 est exclu
- Mettre à jour (NodeSource), puis vérifier :

```
curl -fsSL https://deb.nodesource.com/  
  setup_22.x | sudo -E bash -  
sudo apt-get install -y nodejs  
node -v                % v22.x attendu
```

2. Install & lancement

- À défaut de npx : install globale, puis lancement

```
npm install -g @deepseek-ai/dsh  
dsh web                % ->  
127.0.0.1:3080
```

Télémétrie — confidentialité

Désactiver dans `~/.dsh/profiles/web/cordis.patch.yml`

Ajouter ce bloc, puis lancer `dsh web` :

```
- id: session-telemetry-otel
  name: '@deepseek-ai/dsh-session-telemetry-otel'
  config:
    mode: DISABLED
```

Ordre à respecter : Node $\geq 22,19$ → l'install → `dsh web` — à préparer sur la machine avant la démo B.

Démo B — l'agent local de bout en bout (en direct)

Script

```
# terminal 1 : llama-server (partie 2)
llama-server -hf ggml-org/Qwen3.5-0.8B-
GGUF \
--host 127.0.0.1 --port 8080 \
-c 16384 --jinja

# terminal 2 : le harnais
npx @deepseek-ai/dsh web
# -> UI http://127.0.0.1:3080
# -> provider local (cf. slide avant)

# dans l'UI :
> Lis parts.csv et donne-moi
la quantité totale.
```

À regarder

- l'appel d'outil visible dans l'UI, puis la réponse « 15 »
- le terminal llama-server : des prompts à 10K+ tokens
- le coût du catalogue d'outils → pourquoi un petit modèle rame en agent (mesuré : 2,2 tok/s CPU ⇒ tour d'agent jamais fini)
- plan de secours : basculer sur l'API hébergée, même harnais

Sécurité et garde-fous

- Le composant piloté est **non déterministe** : un agent peut inventer une action, ou déclarer fini ce qui ne l'est pas
- D'où les garde-fous d'un harnais : **approbations** avant les actions sensibles, **sandbox** de fichiers, périmètre de permissions
- **Injection de prompt** : un fichier lu peut contenir des instructions malveillantes → d'où le tri outils/droits
- Réflexes : lire ce que l'agent demande, versionner le travail avec Git (cf. formation d'hier), ne jamais mettre de secrets dans le contexte

Exercice : dimensionner avant d'exécuter

Consigne (5 min en binôme)

Pour chacune des trois machines ci-dessous, choisir **modèle + quantification + contexte** et justifier en une ligne.

Règle : poids $\approx$ paramètres $\times$ bits/poids, + KV cache $\leq$ mémoire disponible.

Machine 1

Laptop de 2017 : 16 Go de RAM, **pas de GPU**

Machine 2

PC gamer : RTX 3060 **12 Go** + 32 Go RAM

Machine 3

MacBook Air M1 : **8 Go** unifiés

Corrigé (indicatif)

- **M1** (CPU seul, 16 Go) : 0,8B–4B en Q4_K_M, contexte modeste (4–8k) — pour du chat; la boucle agent y est pénible (préfilling long à chaque étape)
- **M2** (GPU 12 Go) : 8–9B Q4_K_M en VRAM (~5 Go) + 16k de contexte; ou un MoE moyen partiellement déchargé — agent utilisable
- **M3** (8 Go unifiés) : 3–4B Q4 (~2,5 Go) + KV cache ~2 Go, Metal — chat correct, agent léger
- Dénominateur commun : **le KV cache et le contexte se payent en plus**, et les outils coûtent des tokens — d'où la taille de contexte généreuse pour un agent

Devoir (optionnel) : refaire les démos A et B chez vous, et rapportez tok/s + le modèle que vous pouvez vraiment utiliser en agent.

À retenir

- **Prédiction du token suivant + attention** (requêtes, clés, valeurs) : ça explique à la fois la magie et les limites (coût du contexte, hallucinations)
- **Local = problème de budget** : $\text{paramètres} \times \text{bits/poids} + \text{KV cache} \leq \text{mémoire}$
— Q4_K_M comme défaut sain
- `agent` = modèle + harnais : outils, état, contexte, garde-fous — llama.cpp exécute le modèle, dsh exécute l'agent
- **Un seul protocole partout** (API compatible OpenAI) : n'importe quel runner se branche sur n'importe quel harnais

Ressources : leçon « attention » de 3Blue1Brown • llama.app / docs llama.cpp • docs DeepSeek Harness • Andrej Karpathy (« Let's build GPT ») • r/LocalLLaMA

Merci pour votre attention !

Des questions ?

© ⓘ Ⓢ Ⓢ Formation Robotech — Licence CC BY-NC-SA 4.0
Figures : 3Blue1Brown, 3blue1brown.com/lessons/attention